Security Assurance for Modern Big Data APIs

Enea Manzi[1,*], Marco Anisetti[1]

[1]SESAR Lab, Dipartimento di Informatica "Giovanni Degli Antoni", Università degli Studi di Milano, Via Celoria 18, Milano, 20133, Italia

Abstract

Modern Big Data systems increasingly deliver their value through pipelines composed of services distributed across microservice architectures, and every access to their data and processing capabilities crosses a REST API, typically exposed through a gateway. Assurance of Big Data trustworthiness has so far been addressed mainly at the infrastructure and pipeline levels, while the interface level - where authentication, authorization, and data exposure are actually enforced - remains comparatively overlooked. Automated API testing compounds the gap: it has traditionally focused on functional correctness, and the few security-oriented tools operate at the syntactic level of requests and responses, unable to detect vulnerabilities tied to the target's semantic behavior. Four of the top five categories in the OWASP API Security Top 10 (2023) belong to this class, and deriving evaluation criteria for them from security semantics remains one of the least explored challenges in the literature. Existing solutions further trade portability off against depth, and non-deterministic execution undermines reproducibility.

This work proposes a contract-driven approach to the security assurance of Big Data service APIs. Leveraging the target's OpenAPI specification, the framework generates syntactically valid probes and derives evaluation criteria from the semantics of the assessed controls, embedding them into deterministic test oracles organized into an eight-domain taxonomy that scales coverage with the privileges available during testing. The approach was validated on Forgejo 14.0.3 deployed behind Kong 3.9, instantiating the repository service through which Big Data pipelines are developed and delivered. Results show that the framework correctly classified all test cases, revealed a real discrepancy between the specification's declared requirements and the target's actual behavior, and produced identical outputs across independent runs.

Overall, this work contributes an implementation-agnostic eight-domain taxonomy for the security assurance of Big Data service interfaces, a systematic approach to the oracle problem grounded in security semantics, and a deterministic, DAG-based assessment engine ready for integration into automated delivery pipelines.

Keywords

Big Data assurance, API security, REST, contract-driven testing, test oracles, reproducibility

1. Introduction

Modern Big Data systems deliver their value through pipelines composed of distributed services – ingestion, storage, processing, serving – rather than through a single monolithic engine. Trustworthiness in this setting cannot be a property of one layer alone: it must hold at the pipeline level, at the level of the ecosystem of services executing it, at the level of the data it governs, and at the interface level, and it must hold continuously, as pipelines evolve, rather than as a one-off certificate issued at deployment time.

Existing work on Big Data assurance addresses the first two levels. Anisetti et al. propose an assurance process for Big Data pipelines and their underlying ecosystem [1]; Ardagna et al. address the same problem through service-level agreements [2]; continuous verification across the pipeline life cycle has been framed as a DevSecOps activity [3]; and access control has been enforced at data-ingestion time, an approach recently extended into an attribute-based data engine [4, 5]. Across this body of work, the interface level is the least covered.

Yet the interface is where enforcement actually happens. Every access to a Big Data pipeline and to the data it processes crosses an API, typically exposed through a gateway that concentrates authentication,

ITADATA2026: The 5th Italian Conference on Big Data and Data Science, November 10–12, 2026, Bari, Italy
*Corresponding author.
enea.manzi@studenti.unimi.it (E. Manzi); marco.anisetti@unimi.it (M. Anisetti)
https://eneamanzi.github.io/ (E. Manzi); https://anisetti.di.unimi.it/ (M. Anisetti)
0009-0008-9490-1382 (E. Manzi); 0000-0002-5438-9467 (M. Anisetti)

authorization, and rate limiting; a guarantee proven at the pipeline level does not propagate end-to-end if the API exposing that pipeline is permeable.

Automated API testing does not close this gap. A systematic review of 92 contributions shows that 72 target functional conformance and only 8 address security testing explicitly [6]; and even where security is the explicit target, building the evaluation criteria that decide whether an observed behaviour violates a security guarantee remains the least solved problem in test automation [7].

This work addresses that gap. We propose a contract-driven approach to the security assurance of Big Data service APIs, in which the target's OpenAPI specification governs the construction of syntactically valid probes and the evaluation criterion of each test is derived from the domain knowledge of the security control being exercised, rather than from the syntax of the request or response alone. The approach is organised around three research questions:

- **RQ1**: Can security guarantees of an API-exposed Big Data service be verified beyond syntactic conformance, using criteria derived from the semantics of the security controls themselves?
- **RQ2**: Can such verification be made deterministic and reproducible across independent executions, so that it can be embedded in a continuous release cycle?
- **RQ3**: Does the resulting approach correctly distinguish enforced from bypassed guarantees on a real target, including anomalies not anticipated in advance?

The contributions of this paper are threefold: (i) an implementation-agnostic eight-domain taxonomy for the security assurance of service APIs, with privilege-aware coverage; (ii) a systematic use of specified oracles in API security testing, with evaluation criteria derived from the semantics of each control; and (iii) a deterministic, DAG-based assessment engine whose output is reproducible and machine-actionable, together with its empirical validation.

The remainder of the paper is organised as follows. Section 2 discusses related work. Section 3 details the reference scenario. Section 4 presents the contract-driven approach. Section 5 reports the experimental evaluation. Section 6 draws our conclusions and outlines future work.

2. Related Work

Two lines of work meet in this paper: assurance of Big Data systems, and automated testing of REST APIs.

Assurance of Big Data systems. The trustworthiness of Big Data computations has been addressed as a property of the pipeline, of the ecosystem of services executing it, and of the data it governs. Anisetti et al. propose an assurance process that jointly evaluates a Big Data pipeline and its underlying ecosystem, annotating pipeline templates with non-functional requirements and computing an aggregate trustworthiness confidence level, validated on a Big-Data-Analytics-as-a-Service platform [1]. Ardagna et al. address the same problem through service-level agreements, negotiating and monitoring contractual guarantees, such as data integrity and availability, between the parties of a Big-Data-Analytics-as-a-Service deployment [2]. Continuous verification across the pipeline life cycle has been framed as a DevSecOps activity, embedding assurance checks for non-functional requirements at every stage from planning to operation [3]. At the data governance level, access control has been enforced at ingestion time through data annotations and secure transformations [4], an approach recently extended into an attribute-based data engine that balances data protection against analytics quality [5]. These contributions target the pipeline, the ecosystem executing it, and the data governed within it; none addresses the REST interface through which every access to that pipeline and that data actually takes place, an assumption on which their own guarantees implicitly rely. That assumption is what we make verifiable.

Table 1
What each contract-driven approach checks, and where its oracle stops.

Approach	What it checks	Oracle boundary
RESTler [9]	Server crashes	Only HTTP 500; no semantic check
Atlidakis et al. [10]	Use-after-free, resource leak, hierarchy and namespace violations	Fixed rules, no config. adaptability
NAUTILUS [11]	SQL/command injection, XSS, privilege escalation	Injection-focused only
Schemathesis [12]	Schema conformance, undeclared status codes	Structural, no security check
AGORA/AGORA+ [13, 14]	Functional conformance (mined invariants)	Contract only, no security
SATORI [15]	Functional conformance (static, via LLM)	Contract only, no security
Sahin et al. [16]	Authorization/authentication violations, exposed stack traces, undeclared endpoints	Application layer only, no gateway

Automated testing of REST APIs. A systematic review of 92 contributions published between 2009 and 2022 [6] shows that the field concentrated on system testing, understood as verifying that the API responds correctly and respects the declared formats: 72 of the 92 papers fall in this category. Security testing is addressed explicitly by 8 papers and listed as an open challenge by 10 more. Du et al. quantify the consequence: across stateful REST fuzzers, only 2–5% of the detected HTTP 500 faults correspond to actual security vulnerabilities [8].

The contract-driven line replaces syntactic heuristics with knowledge of the formal specification. RESTler infers producer–consumer dependencies from the OpenAPI specification and generates stateful request sequences; its oracle is the HTTP 500 status code, which the authors acknowledge cannot reach vulnerabilities that do not manifest through status codes [9]. Atlidakis et al. extend it with four formal security rules and active property checkers [10]; NAUTILUS orients the paradigm towards injection-class vulnerabilities requiring ordered operation sequences [11]; Schemathesis derives a structured set of conformance properties from the specification and from RFC 7231 [12]. A distinct direction automates the construction of the oracle itself, dynamically from observed request/response pairs [13, 14] or statically from the specification [15]; both operate in the domain of functional conformance. Closest to our objective, Sahin et al. propose nine automated oracles for authorization, authentication, and injection faults, applied as a post-processing phase over an existing fuzzer [16]; the approach is confined to the application layer and does not cover the configuration of the gateway that fronts the service. Table 1 summarises how these approaches use the specification and where their oracle stops.

The approaches above already expose two of the four gaps that motivate this work. The first is a syntactic blindness: what each tool checks, summarised in Table 1, is either a status code, a structural property of the response, or a fault class mined or authored independently of the target's security semantics; none of them evaluates whether a syntactically valid request violates the access rules the system is supposed to enforce. The second, most structural, is the absence of evaluation criteria for security guarantees: deciding automatically whether an observed behaviour violates such a guarantee requires a criterion that the interface specification alone cannot supply, since it depends on the domain knowledge of the control being exercised. Barr et al. survey this as the oracle problem, and identify the automated construction of such criteria as the bottleneck of test automation [7]. AGORA/AGORA+ and SATORI build oracles explicitly, but mine or infer them from observed traffic or from the contract alone, targeting functional conformance rather than security semantics; Sahin et al. come closer, authoring nine fixed oracles for authorization, authentication, and injection faults, but the result is a closed set confined to the application layer, not a methodology that extends to new domains or to the gateway's

own configuration.

Two further gaps remain. Tools that are portable across targets use the specification to generate syntactically valid requests but not to judge whether responses respect the declared security contract, while tools that reach semantic depth are anchored to a single platform. And the absence of deterministic execution prevents results from being compared across successive runs, which is the precondition for embedding verification in a continuous release cycle.

3. Reference Scenario

We consider the development and delivery of Big Data pipelines. In this scenario, pipeline code, deployment descriptors, build artefacts, and the webhooks that trigger orchestration are managed by a *repository service* that data engineers from different tenants use to upload, version, and activate their pipelines. The service is exposed as a REST API behind an API gateway, which concentrates authentication, authorization, and rate limiting on a single, inspectable configuration plane and constitutes the boundary between clients and internal services [17].

This service belongs to the trust perimeter of the pipelines it hosts, and its API is where that perimeter is enforced. A broken object-level authorization check on that API exposes the code and configuration of another tenant's pipeline; an unverified webhook lets an unauthorised party trigger a pipeline execution; an endpoint that responds without credentials while the published specification declares it protected invalidates the assumptions that consumers of that service legitimately make. None of these failures is visible to an assessment conducted one level below, on the pipeline or on the infrastructure: the requests that realise them are syntactically impeccable, and four of the top five categories of the OWASP API Security Top 10 belong to this class [18]. NIST SP 800-228 formalises the same limit for cloud-native APIs, observing that payload-level defences cannot operate at the semantic level of the interface, and prescribing a formal specification with declared request and response schemas as a precondition for any automated verification [19].

The assurance problem we address is therefore the following: given a service of this kind, described by an OpenAPI specification and exposed through a gateway, decide automatically and reproducibly whether the security guarantees its contract declares are actually enforced.

4. Contract-Driven API Assurance

4.1. Design Principles

Four principles govern the approach.

Application agnosticism. All knowledge of the target is derived at runtime from two sources only: the OpenAPI specification and a configuration file that supplies deployment-specific values, such as concrete values for parameterised paths and credentials for each configured role. No reference to any specific system is hardcoded, so the approach applies to any documented REST API without modifying code and without access to the source of the target.

Contract-driven probing. The specification is the contractual foundation of the whole process. It governs the construction of requests that respect the declared types, formats, and constraints, so that probes are not rejected by the input validation layer before reaching the application logic; it delimits the assessment surface to the documented endpoints, which makes any endpoint responding outside that set a shadow API by definition; and it supplies the primary structural criterion against which responses are evaluated. Using a formal specification to build valid requests and using it to judge responses are distinct capabilities, and existing approaches largely stop at the first [9].

Configuration as the single source of variation. Every operational parameter – timeouts, thresholds, priority filters, output paths – resides in the configuration file; credentials reside in a separate, non-versioned file and are interpolated before parsing. If behaviour changes between two executions, it is because the configuration changed.

Table 2
The eight assurance domains.

Domain	Title	Scope of verification
D0	API Discovery	Inventory of exposed endpoints; detection of shadow APIs not documented in the specification.
D1	Identity & Authentication	Caller recognition mechanisms; robustness of credential transport; JWT life cycle and revocation.
D2	Authorization	Access control logic; horizontal and vertical segregation defects (RBAC, BOLA).
D3	Data Integrity	Structural coherence of payloads; implementation of cryptographic signatures (HMAC).
D4	Availability & Resilience	Defences against resource exhaustion; audit of circuit breakers and rate-limiting thresholds.
D5	Observability	Quality and timeliness of the telemetry the system produces.
D6	Configuration & Hardening	Hardening of the gateway configuration; routing policies; HTTP security headers.
D7	Business Logic & Sensitive Flows	Complex semantic flaws: process constraint evasion, SSRF, race conditions on critical operations.

Deterministic reproducibility. The same target, in the same configuration, must produce identical results at every execution. This is the property that turns an assessment from a single observation into a comparable measurement, and it is a precondition for the empirical claims of Section 5.

4.2. An Eight-Domain Assurance Taxonomy

Security guarantees are organised in eight domains, reported in Table 2, aligned with the risk categories and runtime controls that NIST SP 800-228 identifies for cloud-native APIs [19]. The taxonomy is independent of any implementation: it partitions the security surface of an API exposed through a gateway into semantically distinct categories, each with its own evaluation criteria and priorities, and it is applicable to any structured assessment regardless of the tooling employed.

The first three domains form a chain of preconditions. Verifying authorization (D2) without having established that authentication works (D1) collects evidence about a system whose behaviour under compromised authentication is unknown, and verifying authentication without knowing the attack surface (D0) leaves that evidence incomplete. Domains D3–D7 belong to distinct semantic families with no mutual ordering constraint. Domain D5 (Observability) is architecturally provisioned but carries no active tests in the implementation evaluated here; its numbering is reserved to preserve the coherence of the sequence.

4.3. Specified Oracles for Security Guarantees

Generating inputs automatically is a largely solved problem; deciding whether the observed behaviour is correct is the structural bottleneck of test automation. Barr et al. formalise a test oracle as a partial function $D : T_A \to \mathbb{B}$ mapping a sequence of activities to a verdict, and distribute automation techniques over four categories: *specified* oracles, derived from formal specifications; *derived* oracles, inferred from artefacts such as previous executions; *implicit* oracles, covering universal anomalies such as crashes; and *no-oracle* strategies [7].

We adopt *specified oracles* defined a priori for every individual test and derived from the domain knowledge of the security control being exercised. The methodological procedure is uniform: the class of verification is analysed, the relevant failure conditions are identified, and the criterion is encoded in the logic of the corresponding test. What varies is the source from which the criterion is drawn, and that source is determined by the control, not by the target. Table 3 illustrates this mapping with representative tests drawn from across the eight domains; the same procedure applies uniformly to

Table 3
Sources of the evaluation criterion, by class of guarantee.

Class of guarantee	Source of the criterion	Verdict rule (example)
Attack-surface conformance	Set difference between responding and documented endpoints	an endpoint that responds without being declared is a shadow API
Response structural conformance	Contractual schema declared in the specification	a field present in the response but absent from the declared schema is a violation
Protocol-level requirements	Published technical standards	a deprecated endpoint returning neither `410 Gone` nor a `Sunset` header is a violation
Authentication enforcement	`security` markers of the specification, plus credential-less probing	a `2xx` response to an unauthenticated request on an endpoint declared as protected is a bypass
Authorization	Comparison of responses obtained with credentials of distinct roles	a resource of one role readable with the credentials of another is a violation
Infrastructural guarantees	Direct inspection of the gateway configuration plane	the absence of a required security header (e.g. HSTS) in the gateway's declared configuration is a violation

every test in the suite.

Two consequences follow, one operational and one epistemic.

Operationally, a fixed criterion makes the verdict invariant: the same system, interrogated in the same configuration, always yields the same outcome. This is what makes assessments comparable across successive executions and grounds the reproducibility claim empirically rather than by assertion. To keep that invariance from producing false positives, every HTTP probe is classified under a three-valued encoding rather than a binary one: ENFORCED, the control works; BYPASSED, the control is absent or misconfigured; and INCONCLUSIVE, the response cannot be interpreted with sufficient certainty to justify a finding. A `404` on a parameterised path, for instance, may mean that the gateway rejected the request, that the path does not exist for the value used in the probe, or that the endpoint is genuinely unprotected but the path was wrong; treating it uniformly as a failure would make the assessment unusable.

Epistemically, the correctness of the verdicts is conditional on the correctness of the contract taken as reference. A specification that is incomplete reduces the observable surface without the process being able to detect it; a specification that is contractually wrong produces evaluations grounded in a wrong oracle. We return to this limit in Section 5.6, and Section 5.4 reports a concrete instance of it.

4.4. Privilege-Aware Coverage

Some guarantees are physically unreachable without a specific level of privilege, and that inaccessibility – not a retroactive label – is what determines the visibility level of a test. Perimeter controls that the gateway applies to any interlocutor presuppose no knowledge of the internal infrastructure, and executing them without credentials is the only way to obtain evidence uncontaminated by implicit privileges: these are BLACK_BOX tests, requiring network access alone. An RBAC authorization control requires the system to be in an authenticated state under at least two distinct role identities, because the defect emerges only from comparing what two identities obtain on the same resource: these are GREY_BOX tests. Guarantees that reside in the static configuration of the gateway are verifiable only by direct inspection of its control plane, since no response on the exposed endpoints carries that information: these are WHITE_BOX tests.

Each test declares its strategy as fixed metadata, so assessment depth adapts to the preconditions actually available in a given organisational context without requiring different versions of the tool: the variation lives in the configuration file, not in a mode switch.

4.5. Deterministic Assessment Engine

The assessment unfolds in seven sequential phases with differentiated error semantics. The first four are blocking: initialisation, which validates the configuration and resolves credentials; retrieval and validation of the OpenAPI specification, which builds the map of exposed endpoints; construction of the execution contexts, which assembles the immutable target knowledge and the mutable runtime state; and discovery and scheduling of the tests, which resolves declared dependencies into an execution order. A failure in any of these four halts the process before any test runs. The last three are non-blocking: execution, which runs each test in the scheduled order; teardown, which reverses the resources any test created in last-in-first-out order; and reporting, which aggregates outcomes into machine-readable artefacts. An error in any of them is isolated as a structured result, and the process continues.

Tests declare their dependencies explicitly, and a scheduler resolves them topologically into batches, each grouping the tests whose prerequisites were satisfied in the preceding batches. This structure does more than establish an order: its purpose is semantic correctness. A test that verifies token revocation presupposes that authentication has already been confirmed and a valid token acquired, and without a declared dependency the test's outcome would depend on execution order rather than on the target's actual behaviour. Cycles in the declared dependencies are detected statically, before any test runs, and block the assessment before it starts. A separate stall guard protects against a rarer failure mode: even in an acyclic graph, the scheduler can reach a state where tests remain but none of them is ready; when this happens, the guard reports the exact set of tests that could never be scheduled, rather than failing with a generic error.

Determinism is obtained by the conjunction of four mechanisms: a configuration that fixes every decisional parameter, so that the process has no endogenous sources of variation; a topological resolution that always produces the same batch sequence; a lexicographic ordering of the tests within each batch, which is the only source of a total order among tests that are ready at the same level; and the fixed oracles of Section 4.3. Timestamps and record identifiers vary by nature; what must be invariant are the aggregate counters, the per-test verdicts, and the distribution of findings across domains.

Finally, the process terminates with one of four semantically distinct exit codes: `0` when every active test passed or was skipped; `1` when at least one violation was detected; `2` when no violation was detected but at least one test produced an internal error; and `10` on an infrastructural error that prevented execution. Because the codes are semantically distinct, a CI/CD pipeline can route on the exit code alone, without parsing logs: a violation (`1`) calls for analysing and correcting the target, while an infrastructural error (`10`) calls for diagnosing the assessment's own deployment.

5. Experimental Evaluation

5.1. Testbed

The target environment was selected from the coverage matrix of the test suite: we needed an application with a natively generated OpenAPI specification rather than one reconstructed after the fact, endpoints protected by real authentication, at least two distinct privilege levels, parameterised endpoints with resolvable path values, and behaviour that remains stable across successive runs. Forgejo 14.0.3 [20], a self-hosted Git service exposing repositories, users, tokens, and webhooks through a documented REST API, satisfies all of them, and instantiates the repository service of the reference scenario of Section 3.

Forgejo is exposed through Kong 3.9 [21] in DB-less mode, with PostgreSQL 15-alpine as Forgejo's persistence backend; an ephemeral container provisions three accounts at first start – an administrator and two ordinary users – and terminates. All services run in a dedicated Docker network declared in a single Compose file, on a development host, without dedicated hardware. Figure 1 shows the topology. Kong exposes three ports: plain HTTP for the redirect test, HTTPS with a locally generated self-signed certificate as the primary endpoint, and the Admin API supplying configuration data to the `WHITE_BOX` tests. DB-less mode matters for validation: the configuration lives entirely in a version-controlled file mounted read-only and cannot be altered at runtime, which guarantees that the gateway state is

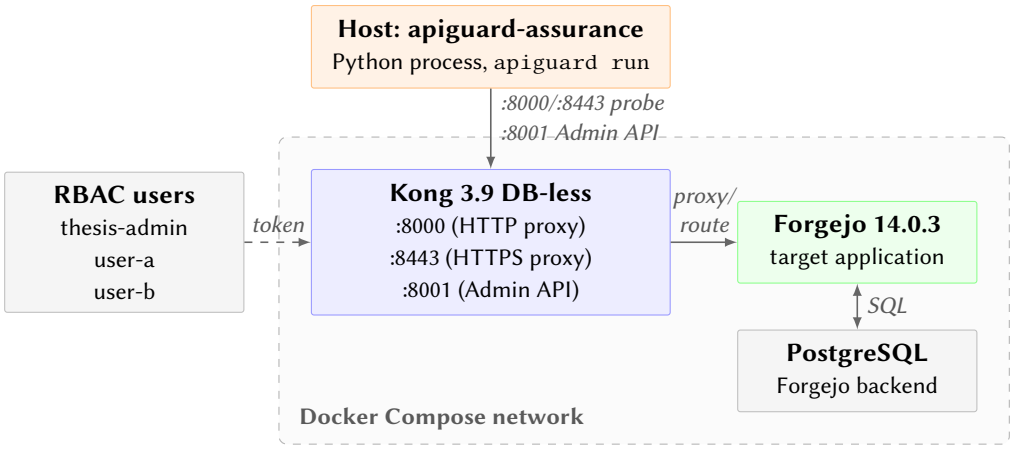

Figure 1: Topology of the experimental testbed: Kong 3.9 routes traffic between the tool and Forgejo 14.0.3, backed by PostgreSQL.

identical at every start.

Because the implementation contains no reference to Forgejo, changing target means updating the configuration file – gateway URL, specification path, path seeds, credentials – not modifying code. This is the concrete demonstration of the application agnosticism claimed in Section 4.

5.2. Experimental Design

Validating an assessment approach against a correctly configured environment would produce a suite of uniform successes: a tautological result that demonstrates nothing about the ability to detect real violations. The testbed is therefore configured so that some tests yield expected failures and others yield expected successes, and the combination constitutes a functionally complete bench on which the approach must distinguish the two.

The testbed combines deliberately engineered conditions with Forgejo's and Kong's behaviour as deployed, which the testbed does not control and cannot predict in advance. Some conditions were deliberately broken to produce expected failures: for instance, Kong's rate-limiting plugin is disabled, a recurrent condition in non-production environments where throttling interferes with application testing, and the TLS certificate is self-signed, which by construction exhibits configurations below production baselines. Others were deliberately configured to produce expected successes: for instance, a response transformer injects the HTTP security headers on every response, and a dedicated service enforces the HTTP-to-HTTPS redirect. The rest of the results follow from Forgejo's and Kong's native behaviour; Section 5.4 discusses the most informative of them.

5.3. Results

The suite comprises 18 active tests across seven domains, executed against the testbed. Table 4 reports the per-test outcome. The assessment produced 9 PASS, 7 FAIL, 2 SKIP, 0 ERROR, and 98 findings, for a pass rate of 56.2%.

Some failures follow directly from the engineered conditions – the rate-limiting test and the two external TLS analysers, which detect one and three findings respectively on the self-signed certificate – while others surface native behaviour of Forgejo and Kong, such as the discovery tests, the SSRF test, and test 1.1, which alone accounts for 74 of the 98 findings, a point we return to in Section 5.4. The distribution of findings across these tests confirms that the analysis discriminates individual weaknesses rather than collapsing them into a single verdict.

The two skips illustrate the behaviour of the process in the absence of applicability conditions. Every test always produces a result: when the applicable scope is empty or a prerequisite is unmet, that result is a SKIP with an explicit reason, not a silent error. The specification of Forgejo declares no

Table 4
Per-test results of the assessment.

Test	Status	Findings	Test	Status	Findings
0.1	FAIL	16	4.1	FAIL	2
0.2	FAIL	1	4.2	PASS	0
0.3	SKIP	0	4.3	PASS	0
1.1	FAIL	74	6.2	PASS	0
1.4	PASS	0	6.4	PASS	0
1.5	PASS	0	7.2	FAIL	1
1.6	SKIP	0	ext.0.1.nuclei	PASS	0
2.1	PASS	0	ext.1.5.sslyze	FAIL	1
3.3	PASS	0	ext.1.5.testssl	FAIL	3
				Total	**98**

deprecated endpoint, so the deprecation test reports a documented skip; the testbed gateway manages no external session store, so the corresponding test does the same. In both cases a documented skip is the semantically correct answer, and is distinct from an ERROR, which would indicate a malfunction of the process itself.

The nine successes likewise mix engineered and native outcomes: the security headers test and the HTTP-to-HTTPS redirect test reflect the deliberately configured conditions, while correct token revocation, authorization enforcement, upstream configuration, and the data integrity and availability controls reflect Forgejo's and Kong's own behaviour operating as intended. The external scanner detected no shadow API, confirming that the exposed surface matches the one documented by the specification.

5.4. A Discrepancy Between Contract and Behaviour

The most informative outcome of the assessment is test 1.1. Forgejo declares an authentication requirement at the document level of its specification; the analyser propagates that marker to every endpoint and classifies them as requiring authentication. When the test interrogates them without credentials and receives 2xx, the three-valued oracle classifies the outcome as BYPASSED and emits a finding, for a total of 74.

The verdict is contractually correct, and the behaviour it exposes is a genuine misalignment: the endpoints in question are genuinely public, but the published contract does not say so. This is precisely the class of anomaly that contract-driven assessment exists to detect, and it is exactly the risk anticipated in Section 3: an endpoint that responds without credentials while the specification declares it protected invalidates the assumptions that data engineers legitimately make about the pipeline repository. A contract that overstates its own protections is as much an assurance defect as an unenforced control, because it leaves every consumer of that contract unable to verify which guarantees actually hold. The case also illustrates concretely the epistemic limit of Section 4.3: findings of this kind are correct with respect to the contract, and require an analyst to decide whether the discrepancy is an intentional configuration or an actual vulnerability.

5.5. Determinism, Idempotence, and Footprint

The assessment was executed twice in sequence against the same testbed, with no manual intervention between the runs. Table 5 reports the aggregate indicators. Finding counters, per-test statuses, exit code, and pass rate are byte-identical across the two runs: the deterministic part of the result exhibits no variation. The wall-clock delta is 0.23%, which falls within the expected fluctuation of HTTP latency towards the target and is not attributable to non-deterministic behaviour of the process.

Teardown is the precondition of idempotence: without a complete cleanup of the resources created during the first execution, the second would encounter an altered state and produce different results.

Table 5
Aggregate indicators over two independent runs, and execution profile.

Indicator	Run 1	Run 2	Outcome
PASS / FAIL / SKIP / ERROR	9/7/2/0	9/7/2/0	identical
Total findings	98	98	identical
Pass rate (%)	56.2	56.2	identical
Exit code	1	1	identical
Duration (s)	290.14	290.81	$+0.23\%$
Residual resources	0	0	identical

Verification after each run confirmed that the teardown phase drained the LIFO registry with no residual resources, leaving no tokens and no repositories created by the assessment on the target accounts.

The execution profile qualifies operational integrability. Of approximately 290 seconds of wall-clock time, the process consumed about 35 seconds of CPU in user space and 41 seconds in kernel space: 26% of the total. The remaining 214 seconds were spent waiting for HTTP responses, for the completion of external analysers, and for disk I/O. The regime is dominated by network latency, so optimising the implementation would have marginal impact; the significant reduction would come from parallelising tests within the same batch that have no mutual dependencies, an extension left to future work.

5.6. Threats to Validity

Four limits bound the scope of these results. First, the correctness of every verdict is conditional on the quality of the specification taken as the oracle: an incomplete contract reduces the observable surface without the process being able to detect the reduction, and a wrong contract grounds correct verdicts on wrong premises. Second, validation was conducted on a single target, selected for its structural properties rather than for affinity with the approach; while the absence of any target-specific code makes portability a property of the design, further validation across other services in a Big Data pipeline – ingestion, processing, and serving components, beyond the repository service evaluated here – would strengthen the generality of the approach. Third, WHITE_BOX coverage depends on access to the configuration plane of the gateway, which is typically unavailable in managed cloud deployments; in those contexts the assessment degrades to its BLACK_BOX and GREY_BOX subsets, with documented rather than silent loss of coverage. Fourth, one of the eight domains carries no active tests in the evaluated implementation.

6. Conclusions

We addressed the security assurance of Big Data services at the level where every access to their data and pipelines is actually enforced: the API. We proposed a contract-driven approach in which the OpenAPI specification of the target governs the construction of syntactically valid probes, while the evaluation criterion of each test is derived from the domain knowledge of the security control being exercised and encoded a priori as a specified oracle. Tests are organised in an eight-domain taxonomy whose coverage scales with the privileges available to the assessor, and are executed by a deterministic engine that resolves dependencies topologically and communicates its outcome through semantically distinct exit codes.

The evaluation, conducted on a service instantiating the repository role of a Big Data pipeline delivery workflow and exposed through an API gateway, answers the three research questions of Section 1: the approach distinguishes correctly between conformance and violation on a testbed configured to yield both, including anomalies not anticipated in advance (RQ3); it surfaces a real misalignment between a published contract and the observed behaviour of the system, using criteria derived from the semantics of the security controls themselves (RQ1); and independent executions produce byte-identical results

(RQ2), which makes the assessment comparable over time and embeddable in a continuous release cycle.

Future work proceeds along two directions. On the operational side, completing the partially covered domains and providing adapters for the managed gateways of the major cloud providers would extend coverage in the deployments where it is currently narrowest. On the research side, the formalisation of security oracles remains open: the approach presented here derives evaluation criteria systematically but manually, and characterising the conditions under which such criteria can be derived from the semantics of a control – rather than authored per control – is the natural continuation of this work.

Acknowledgments

Thanks to the developers of ACM consolidated LaTeX styles https://github.com/borisveytsman/acmart and to the developers of Elsevier updated LaTeX templates https://www.ctan.org/tex-archive/macros/latex/contrib/els-cas-templates.

Declaration on Generative AI

Either:
The author(s) have not employed any Generative AI tools.

Or (by using the activity taxonomy in ceur-ws.org/genai-tax.html):
During the preparation of this work, the author(s) used X-GPT-4 and Gramby in order to: Grammar and spelling check. Further, the author(s) used X-AI-IMG for figures 3 and 4 in order to: Generate images. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

References

[1] M. Anisetti, C. A. Ardagna, F. Berto, An assurance process for Big Data trustworthiness, Future Generation Computer Systems 146 (2023) 34–46. URL: https://www.sciencedirect.com/science/article/pii/S0167739X23001371. doi:10.1016/j.future.2023.04.003.

[2] C. A. Ardagna, N. Bena, C. Hebert, M. Krotsiani, C. Kloukinas, G. Spanoudakis, Big Data Assurance: An Approach Based on Service-Level Agreements, Big Data 11 (2023) 239–254. URL: https://journals.sagepub.com/action/showAbstract. doi:10.1089/big.2021.0369.

[3] M. Anisetti, N. Bena, F. Berto, G. Jeon, A DevSecOps-based Assurance Process for Big Data Analytics, in: 2022 IEEE International Conference on Web Services (ICWS), 2022, pp. 1–10. URL: https://ieeexplore.ieee.org/abstract/document/9885738. doi:10.1109/ICWS55610.2022.00017.

[4] M. Anisetti, C. A. Ardagna, C. Braghin, E. Damiani, A. Polimeno, A. Balestrucci, Dynamic and Scalable Enforcement of Access Control Policies for Big Data, in: Proceedings of the 13th International Conference on Management of Digital EcoSystems, ACM, Virtual Event Tunisia, 2021, pp. 71–78. URL: https://dl.acm.org/doi/10.1145/3444757.3485107. doi:10.1145/3444757.3485107.

[5] A. Polimeno, P. Mignone, C. Braghin, M. Anisetti, M. Ceci, D. Malerba, C. A. Ardagna, Balancing Protection and Quality in Big Data Analytics Pipelines, Big Data 13 (2025) 127–143. doi:10.1089/big.2023.0065.

[6] A. Golmohammadi, M. Zhang, A. Arcuri, Testing RESTful APIs: A Survey, ACM Transactions on Software Engineering and Methodology 33 (2023) 27:1–27:41. URL: https://dl.acm.org/doi/10.1145/3617175. doi:10.1145/3617175.

[7] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo, The Oracle Problem in Software Testing: A Survey, IEEE Transactions on Software Engineering 41 (2015) 507–525. URL: https://ieeexplore.ieee.org/document/6963470. doi:10.1109/TSE.2014.2372785.

[8] W. Du, J. Li, Y. Wang, L. Chen, R. Zhao, J. Zhu, Z. Han, Y. Wang, Z. Xue, Vulnerability-oriented Testing for RESTful APIs, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 739–755. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/du.

[9] V. Atlidakis, P. Godefroid, M. Polishchuk, RESTler: Stateful REST API Fuzzing, in: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), IEEE, Montreal, QC, Canada, 2019, pp. 748–758. URL: https://ieeexplore.ieee.org/document/8811961/. doi:10.1109/ICSE.2019.00083.

[10] V. Atlidakis, P. Godefroid, M. Polishchuk, Checking Security Properties of Cloud Service REST APIs, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), IEEE, Porto, Portugal, 2020, pp. 387–397. URL: https://ieeexplore.ieee.org/document/9159084/. doi:10.1109/ICST46399.2020.00046.

[11] G. Deng, Z. Zhang, Y. Li, Y. Liu, T. Zhang, Y. Liu, G. Yu, D. Wang, NAUTILUS: Automated RESTful API Vulnerability Detection, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 5593–5609. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/deng-gelei.

[12] Z. Hatfield-Dodds, D. Dygalo, Deriving semantics-aware fuzzers from web API schemas, in: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 345–346. URL: https://dl.acm.org/doi/10.1145/3510454.3528637. doi:10.1145/3510454.3528637.

[13] J. C. Alonso, S. Segura, A. Ruiz-Cortés, AGORA: Automated Generation of Test Oracles for REST APIs, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 1018–1030. URL: https://dl.acm.org/doi/10.1145/3597926.3598114. doi:10.1145/3597926.3598114.

[14] J. C. Alonso, M. D. Ernst, S. Segura, A. Ruiz-Cortés, Test Oracle Generation for REST APIs, ACM Transactions on Software Engineering and Methodology 35 (2025) 19:1–19:37. URL: https://dl.acm.org/doi/10.1145/3726524. doi:10.1145/3726524.

[15] J. C. Alonso, A. Martin-Lopez, S. Segura, G. Bavota, A. Ruiz-Cortés, SATORI: Static Test Oracle Generation for REST APIs, in: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025, pp. 1364–1376. URL: https://ieeexplore.ieee.org/document/11334625. doi:10.1109/ASE63991.2025.00116.

[16] O. Sahin, M. Zhang, A. Arcuri, Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks, 2026. URL: http://arxiv.org/abs/2604.00702. doi:10.48550/arXiv.2604.00702. arXiv:2604.00702.

[17] S. Rose, O. Borchert, S. Mitchell, S. Connelly, Zero Trust Architecture, Technical Report, National Institute of Standards and Technology, 2020. URL: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf. doi:10.6028/NIST.SP.800-207.

[18] OWASP Foundation, OWASP Top 10 API Security Risks – 2023, 2023. URL: https://owasp.org/API-Security/editions/2023/en/0x11-t10/.

[19] R. Chandramouli, Z. Butcher, Guidelines for API Protection for Cloud-Native Systems, Technical Report NIST Special Publication (SP) 800-228, National Institute of Standards and Technology, 2026. URL: https://csrc.nist.gov/pubs/sp/800/228/upd1/final. doi:10.6028/NIST.SP.800-228-upd1.

[20] The Forgejo Contributors, Forgejo: Beyond coding. We forge., 2026. URL: https://forgejo.org/.

[21] Kong Inc., Kong Gateway, 2026. URL: https://konghq.com/products/kong-gateway.